EC4219: Software Engineering

Lecture 5 — Problem Solving using SMT Solvers

Sunbeom So
2025 Spring

The Z3 SMT Solver

- SMT: **S**atisfiability **M**odulo **T**heories
  - "Modulo" $\approx$ "in terms of"
- SMT solver: a prover for automatically solving SMT problems
- Z3: A popular SMT solver from Microsoft Research

  https://github.com/Z3Prover/z3

  Supports many useful theories.
  - Propositional Logic, Equality, Uninterpreted Functions, Arithmetic, Arrays, Bit-vectors, etc.
- References
  - Z3 API in Python

    http://ericpony.github.io/z3py-tutorial/guide-examples.htm
  - Z3 API in OCaml

    https://z3prover.github.io/api/html/ml/Z3.html

Goal: understanding how first-order theories can be applied in practice with Z3

Propositional Logic

Consider the formula

$$(p \rightarrow q) \wedge (r \leftrightarrow \neg q) \wedge (\neg p \vee r).$$

The Python code to check its satisfiability:

```
1  from z3 import *
2  p = Bool ('p')
3  q = Bool ('q')
4  r = Bool('r')
5  f = And (Implies (p,q), r == Not (q), Or (Not(p), r))
6  solve (f)
7  # s = Solver ()
8  # s.add(f)
9  # print (s.check())
10 # print (s.model())
```

The Z3 solver will produce a satisfying assignment (if any), e.g.,

$$[\mathtt{q} = \mathtt{False}, \mathtt{p} = \mathtt{False}, \mathtt{r} = \mathtt{True}]$$

Propositional Logic (Cont'd)

- Consider the formula $p \wedge \neg p$.

```
1 from z3 import *
2 p = Bool ('p')
3 solve (And (p, Not (p)))
```

```
$ python3 test.py
no solution
```

- How to check $(p \vee q) \implies (p \wedge q)$?

```
1 from z3 import *
2 p = Bool ('p')
3 q = Bool ('q')
4 solve (Not (Implies (Or (p, q), And (p,q))))
```

```
$ python3 test.py
no solution
```

Arithmetic (Integer, Real)

```
1 from z3 import *
2
3 x = Int ('x')
4 y = Int ('y')
5 solve (x > 2, y < 10, x + 2*y == 7)
6
7 x = Real ('x')
8 y = Real ('y')
9 solve (x ** 2 + y ** 2 > 3, x**3 + y < 5) # **: power
```

```
$ python3 test.py
[y = 0, x = 7]
[x = 1/8, y= 2]
```

Arithmetic (Integer, Real)

The satisfiability results may vary depending on the underlying theories.

```
1 from z3 import *
2
3 x = Int ('x')
4 y = Int ('y')
5 solve (Exists([x], 2*x == 7))
6
7 x = Real ('x')
8 y = Real ('y')
9 solve (Exists([x], 2*x == 7))
```

```
$ python3 test.py
no solution
[]
```

Bit-vector

```
1 from z3 import *
2
3 x = BitVec('x', 4)
4
5 solve ( x > 7) # signed greater-than
6 solve ( UGT (x, 7) ) # unsigned greater-than
7 solve ( x == 16)
8 solve ( x == -16)
9 solve (Not((x == -16) == (x == 16)))
```

```
no solution
[x = 8]
[x = 0]
[x = 0]
???
```

Uninterpreted Functions

```
1  from z3 import *
2  x = Int ('x')
3  y = Int ('y')
4  f = Function ('f', IntSort(), IntSort())
5
6  s = Solver()
7  s.add (f(f(x)) == x, f(x) == y, x != y)
8  print s.check()
9
10 m = s.model()
11 print m
12 print ''f(f(x)) ='', m.evaluate (f(f(x)))
13 print ''f(x) ='', m.evaluate(f(x))
```

```
sat
[x = 0, y = 1, f = [0 -> 1, 1 -> 0, else -> 1]]
f(f(x)) = 0
f(x) = 1
```

Constraint Generation with Python List Comprehension

You can generate variables in a concise way using list comprehensions.

```
1  from z3 import *
2  X = [Int ('x % s', %i) for i in range (5)]
3  Y = [Int ('y % s', %i) for i in range (5)]
4  print X, Y
5
6  X_gt_Y = [X[i] > Y[i] for i in range(5)]
7  print (X_gt_Y)
8
9  f = And (X_gt_Y)
10 solve(f)
```

```
[x0, x1, x2, x3, x4] [y0, y1, y2, y3, y4]
[x0 > y0, x1 > y1, x2 > y2, x3 > y3, x4 > y4]
And(x0 > y0, x1 > y1, x2 > y2, x3 > y3, x4 > y4)
[y4 = 0, x4 = 1, y3 = 0, x3 = 1, y2 = 0, x2 = 1,
 y1 = 0, x1 = 1, y0 = 0, x0 = 1]
```

Problem 1: Program Equivalence

Consider the two code fragments.

```
if (!a&&!b) then h
else if (!a) then g else f
```

```
if (a) then f
else if (b) then g else h
```

where `f`, `g`, and `h` are propositional variables asserting that certain conditions are met.

The latter might have been generated from an optimizing compiler. We would like to prove that the two programs are equivalent in terms of reachable memory states.

Problem 1: Program Equivalence (Cont'd)

The if-then-else construct can be replaced by a PL formula as follows:

$$\texttt{if } x \texttt{ then } y \texttt{ else } z \equiv (x \wedge y) \vee (\neg x \wedge z)$$

The problem of checking the equivalence is to check the validity of the formula:

$$\begin{aligned} F : & ((\neg a \wedge \neg b) \wedge h) \vee (\neg(\neg a \wedge \neg b) \wedge ((\neg a \wedge g) \vee (a \wedge f))) \\ \iff & (a \wedge f) \vee (\neg a \wedge ((b \wedge g) \vee (\neg b \wedge h))) \end{aligned}$$

(Exercise) Write a Python program that checks the validity of F.

Problem 2: Seat Assignment

Consider three people A, B, and C who need to be seated in a row. Suppose we have three constraints:

- A does not want to sit next to C
- A does not want to sit in the leftmost chair
- B does not want to sit to the right of C

We would like to check if there is a seat assignment for the three persons that satisfies the above constraints.

Problem 2: Seat Assignment (Cont'd)

To encode the problem, let $\boldsymbol{X_{ij}}$ be boolean variables such that

$$\boldsymbol{X_{ij}} \iff \text{a person } \boldsymbol{i} \text{ seats in a chair } \boldsymbol{j}$$

Problem 2: Seat Assignment (Cont'd)

To encode the problem, let X_{ij} be boolean variables such that

$$X_{ij} \iff \text{a person } i \text{ seats in a chair } j$$

- A does not want to sit next to C

$$(X_{00} \rightarrow \neg X_{21}) \wedge (X_{01} \rightarrow (\neg X_{20} \wedge \neg X_{22})) \wedge (X_{02} \rightarrow \neg X_{21})$$

- A does not want to sit in the leftmost chair

$$\neg X_{00}$$

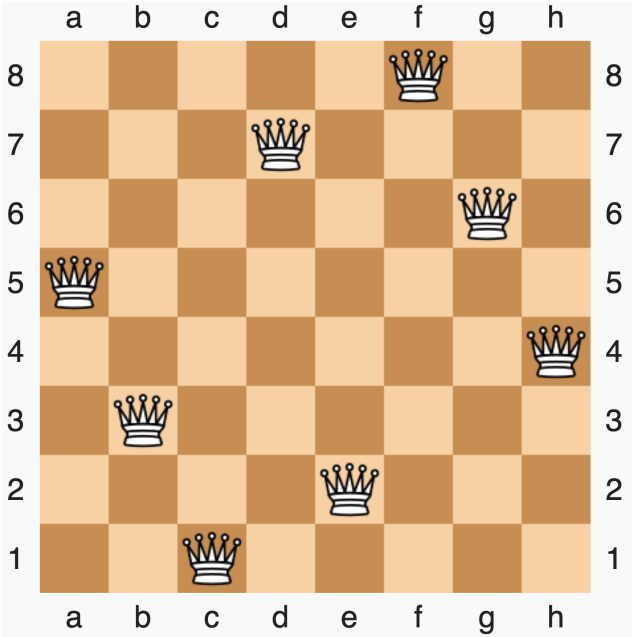
- B does not want to sit to the right of C

$$(X_{11} \rightarrow \neg X_{20}) \wedge (X_{12} \rightarrow \mathit{false})$$

Problem 3: Eight Queens Puzzle[1]

The eight-queens puzzle is the problem of placing eight chess queens on an 8x8 chessboard so that no two queens attack each other. Thus, a solution requires that:

1. No two queens share the same row,
2. No two queens share the same column, and
3. No two queens share the same diagonal.



[1] The image captured from `https://en.wikipedia.org/wiki/Eight_queens_puzzle`

Problem 3: Eight Queens Puzzle (Cont'd)

Constrain variables Q_i, denoting the column of position of the queen in row i.

- Each queen is in the columns $\{1, \cdots, 8\}$.

$$\bigwedge_{i=1}^{8} 1 \leq Q_i \wedge Q_i \leq 8$$

- No queens share the same column.

$$\bigwedge_{i=1}^{8} \bigwedge_{j=1}^{8} (i \neq j \implies Q_i \neq Q_j)$$

- No queens share the same diagonal.

$$\bigwedge_{i=1}^{8} \bigwedge_{j=1}^{i} (i \neq j \implies Q_i - Q_j \neq i - j \wedge Q_i - Q_j \neq j - i)$$

Summary

- Problem solving using Z3
- Try all the examples by yourself!